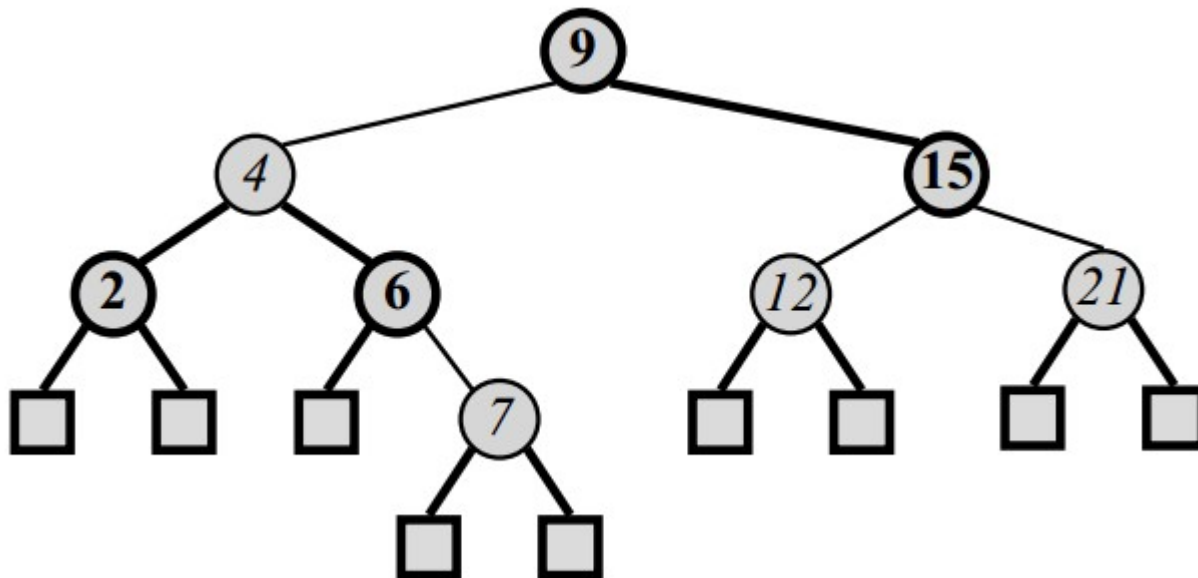


Red-Black Tree

Prof. Siddharth Shah

Red-Black Tree

- Red-Black Tree is a self-balancing Binary Search Tree (BST) where every node follows rules mentioned below:
 - Every node has a colour either **red** or **black**.
 - **Root** of tree is always black.
 - There are no two adjacent **red** nodes (A red node cannot have a red parent or red child).
 - Every path from root to a NULL node has same number of **black** nodes (**black height of tree**).



Red-Black Tree

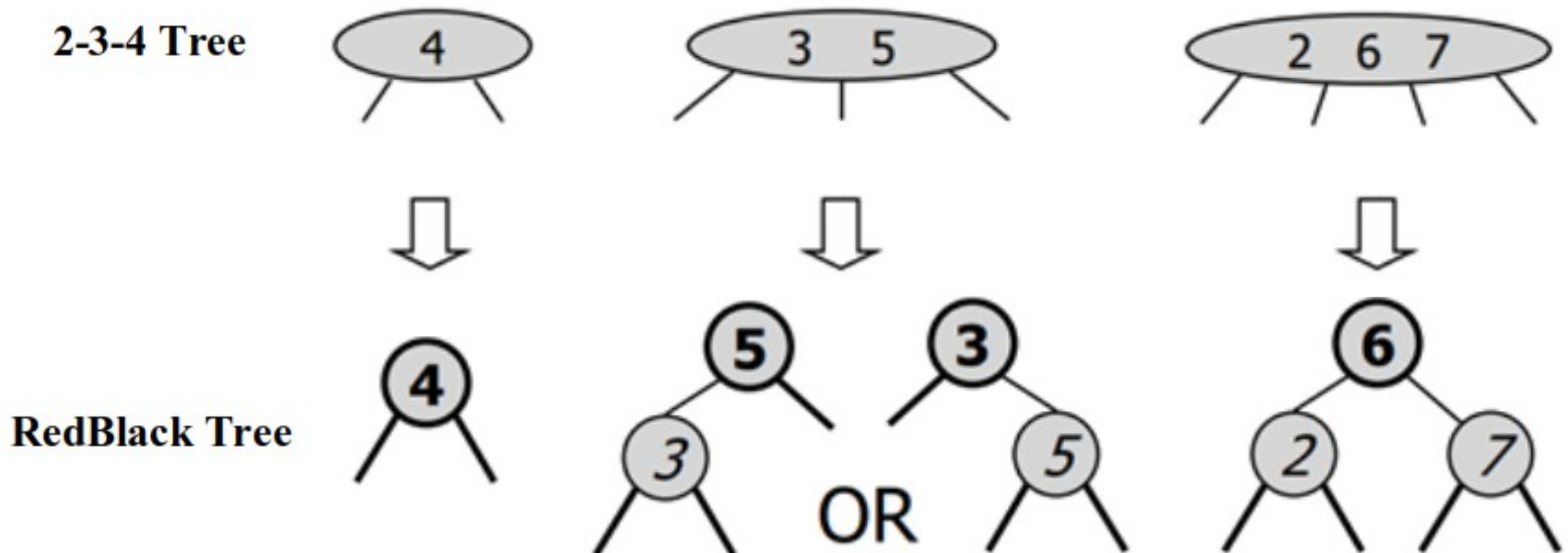
Some more characteristics of Red-Black Tree:

- **NULL** nodes are colored **BLACK**

✓ • Path from root to **farthest leaf** is no more than **twice**
✱ as long as path from root to **nearest leaf**. (Note:
Considering **NULL** node as a leaf)

Relation with 2-3-4 Tree

- A red-black tree is a representation of a 2-3-4 tree by means of a binary search tree whose nodes are coloured red or black.
- In comparison with its associated 2-3-4 tree, a red-black tree has
 - same logarithmic time performance
 - simpler implementation with a single node type



Insertion Operation

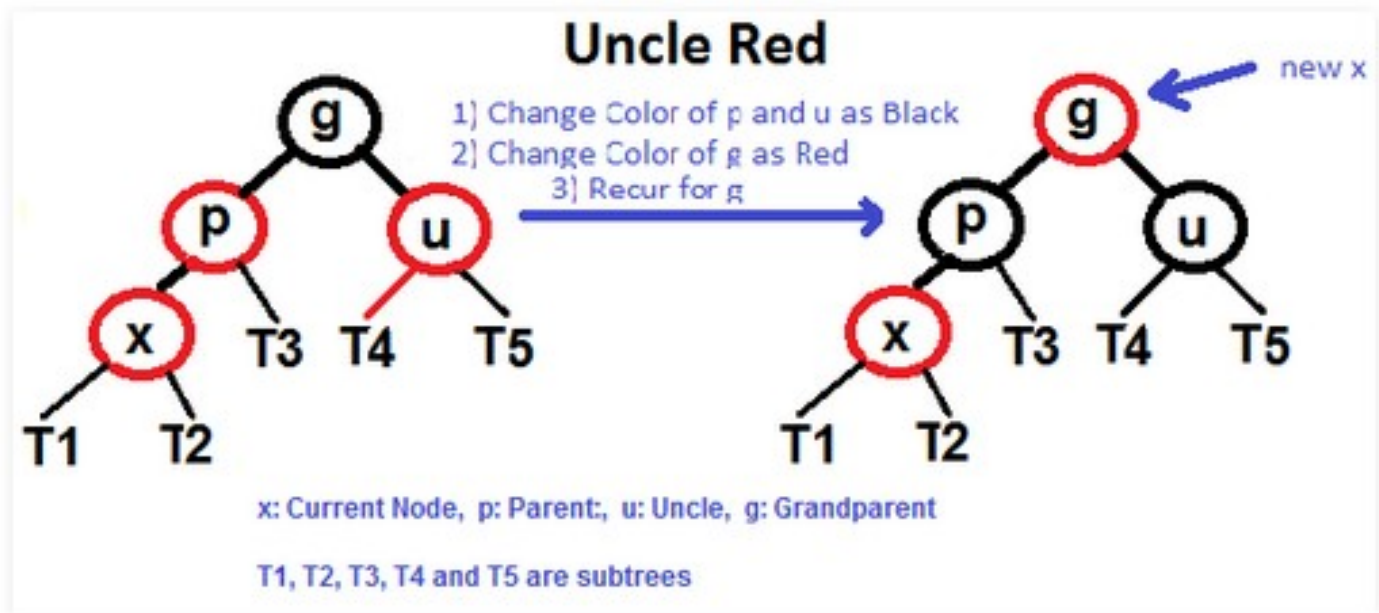
- In Red-Black tree, two tools are used to do balancing, 1) **Recoloring** and 2) **Rotation**.
- Recoloring is performed first, if recoloring doesn't work, then rotation is performed.
- The algorithm has mainly two cases depending upon the color of **uncle**.
 1. If **uncle** is **red**, we do recoloring.
 2. If **uncle** is **black**, we do rotations and/or recoloring.
- Color of a **NULL** node is considered as BLACK.

Insertion Algorithm

1. Let x be the newly inserted node. Perform standard BST insertion and make the color of newly inserted node as **RED**.
2. If x is root, change color of x as BLACK (Black height of complete tree increases by 1).
3. If color of x 's parent is not **BLACK** or x is not root, handle it. There are two cases under this situation:
 - Case 1: x 's uncle is **RED**
 - Case 2: x 's uncle is **BLACK**

Case 1: x's uncle is **RED**

- Change color of parent and uncle as **BLACK**.
- Change color of grand parent as **RED**.
- Change $x = x$'s grandparent, repeat steps 2 and 3 for new x .

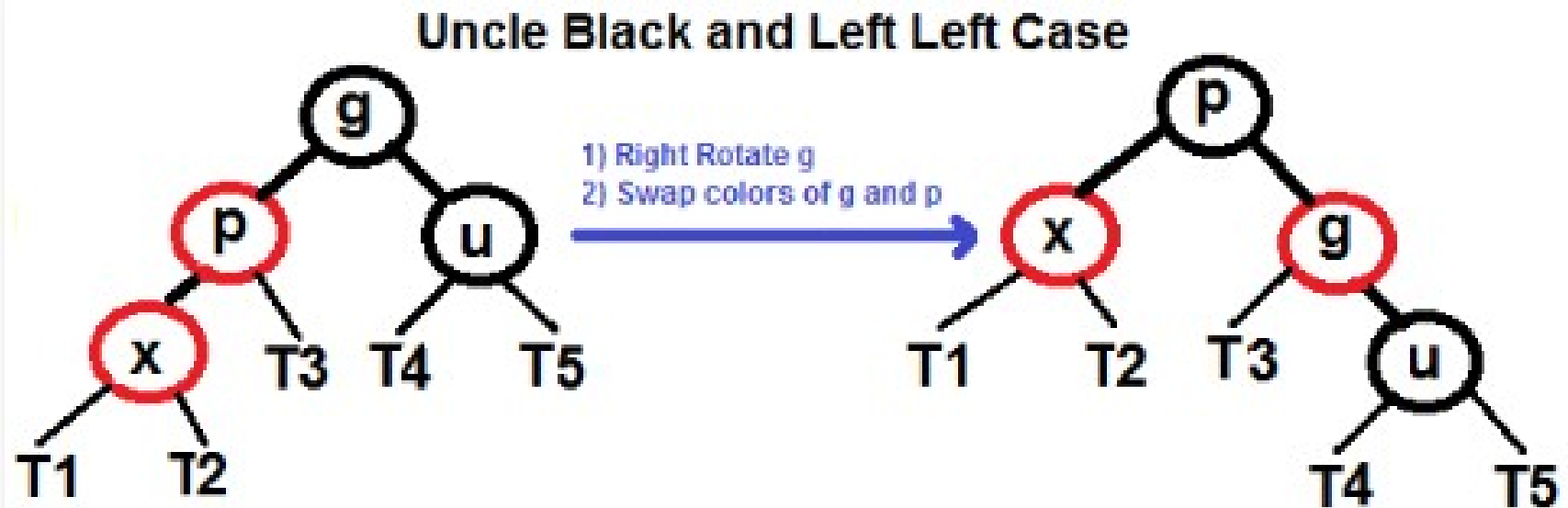


Case 2: x's uncle is **BLACK**

There can be 4 configurations for x, x's parent (p) and x's grandparent (g) similar to AVL Tree.

1. Left Left Case (p is left child of g and x is left child of p)
2. Left Right Case (p is left child of g and x is right child of p)
3. Right Right Case (p is right child of g and x is right child of p)
4. Right Left Case (p is right child of g and x is left child of p)

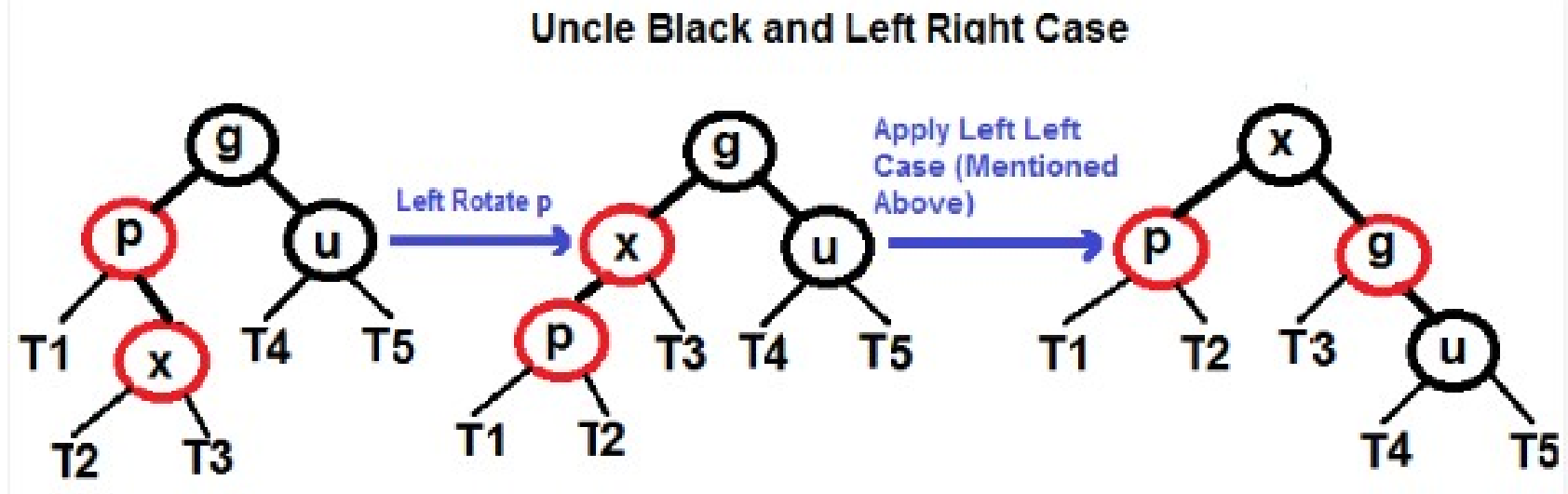
Case 2.1: Left Left Case



x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

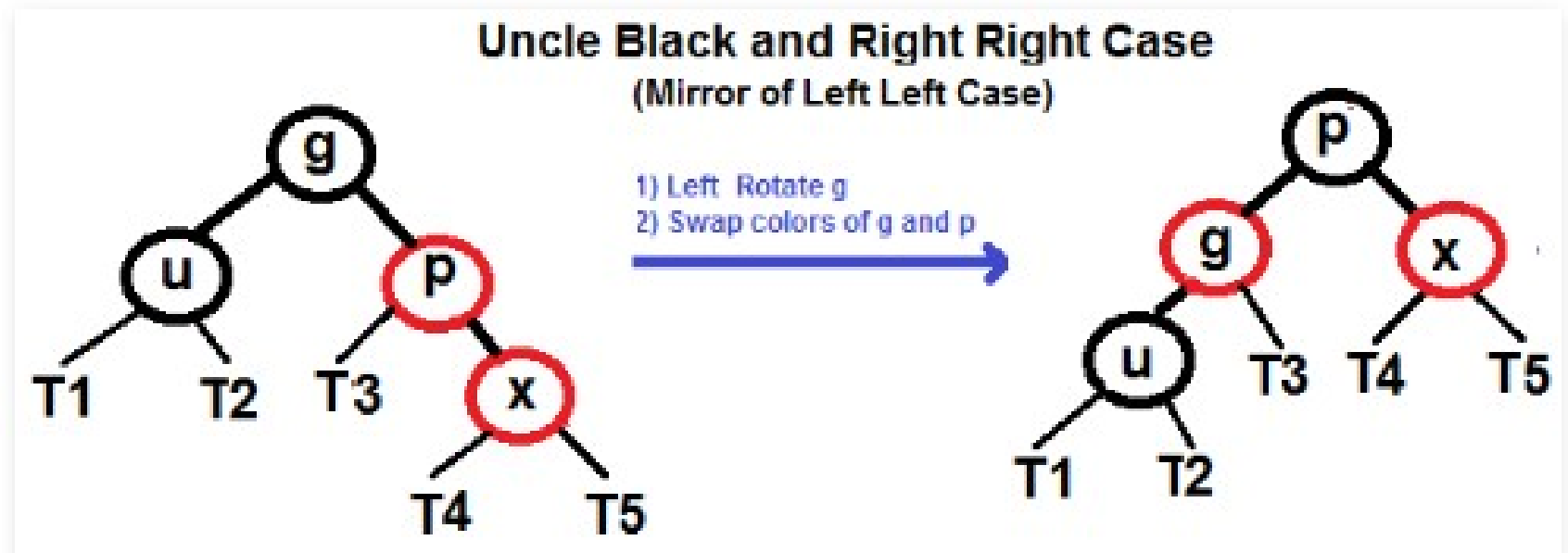
Case 2.2: Left Right Case



x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

Case 2.3: Right Right Case

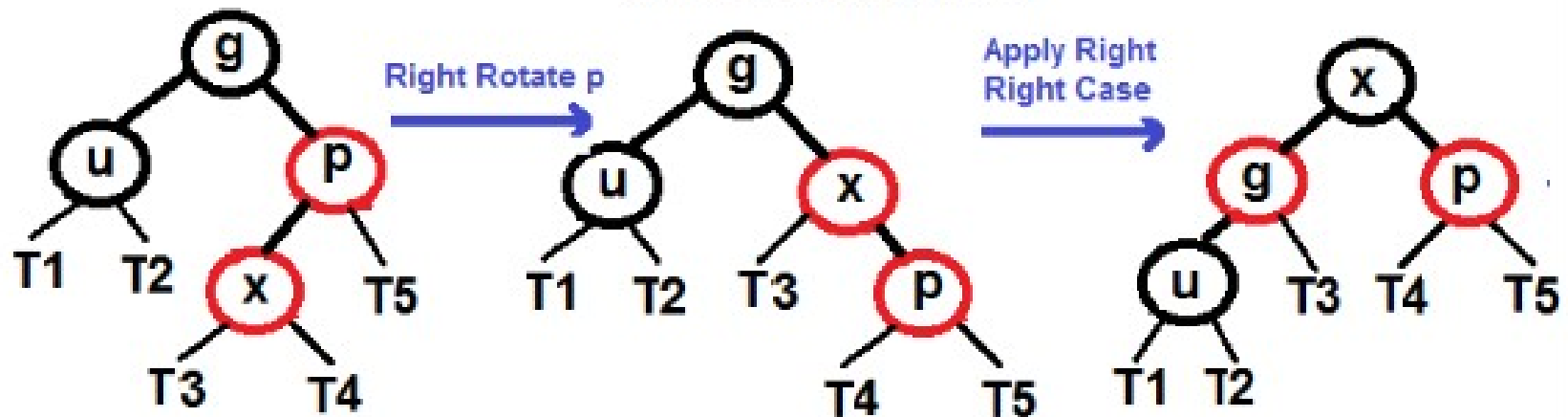


x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

Case 2.3: Right Left Case

Uncle Black and Right Left Case
(Mirror of Left Right Case)

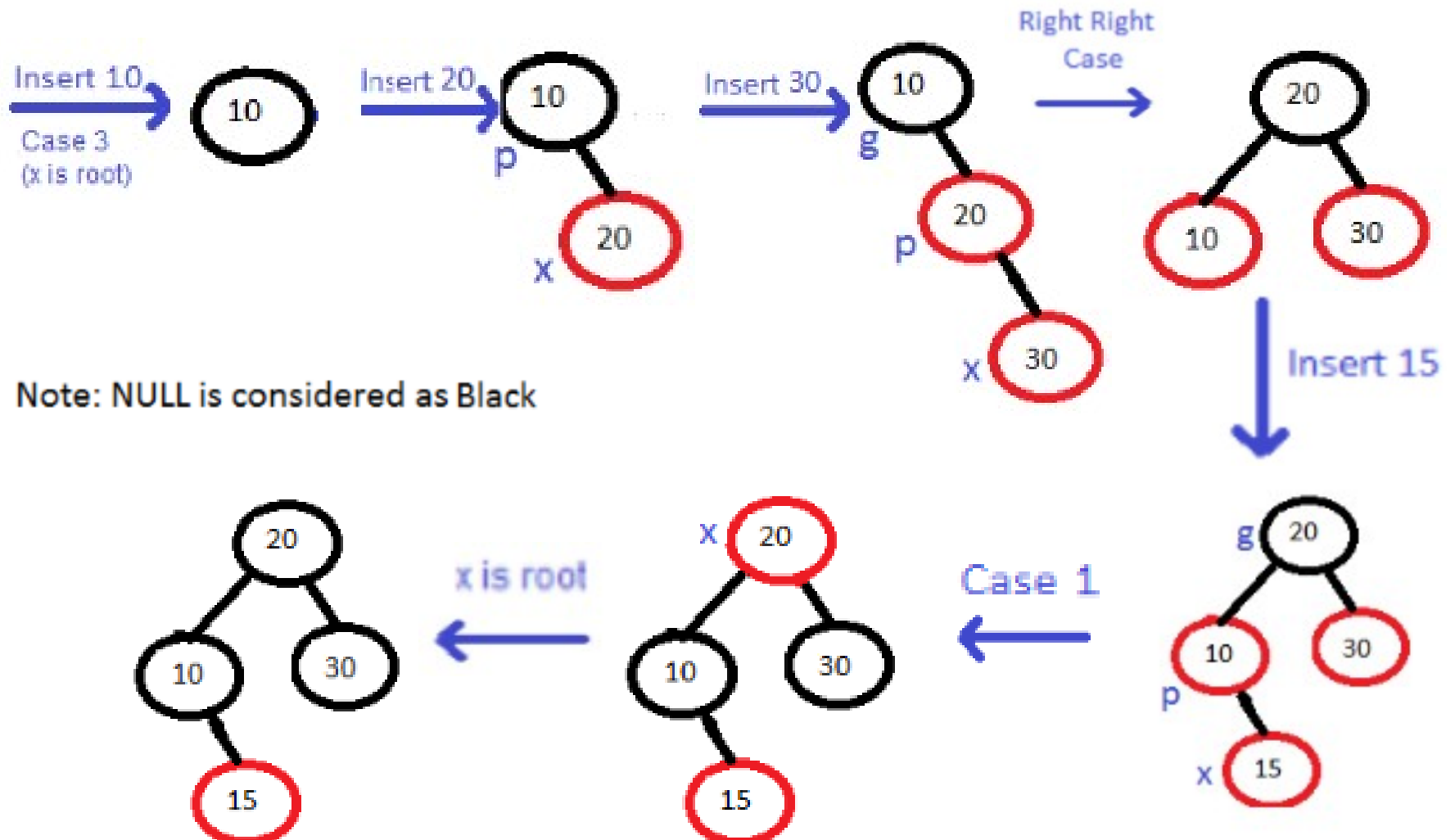


x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

Example 1

Insert **10**, **20**, **30** and **15** in an empty Red-Black Tree.



Example 2

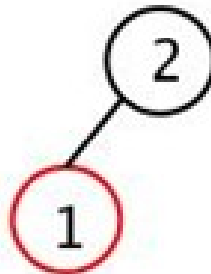
Insert **2**, **1**, **4**, **5**, **9**, **3**, **6** and **7** in an empty Red-Black Tree.

Insert(2)

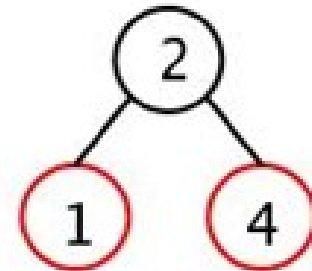


2 is root

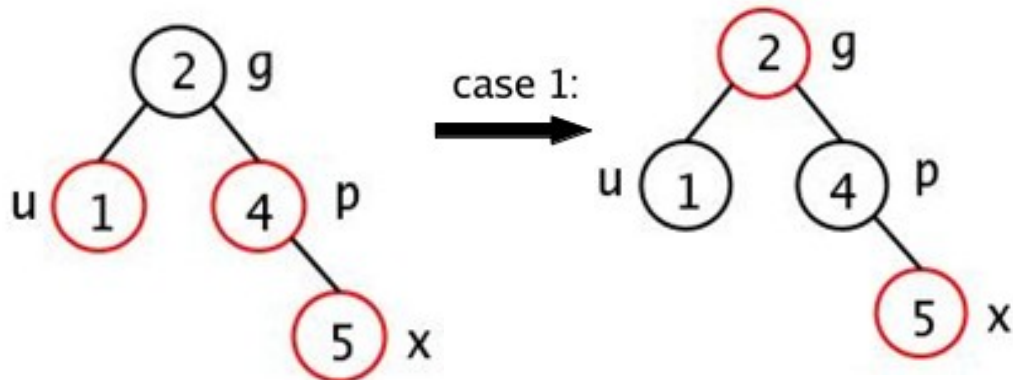
Insert(1)



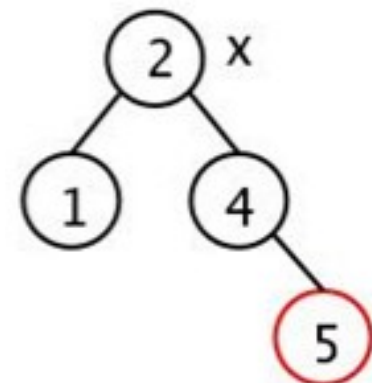
Insert(4)



Insert(5)



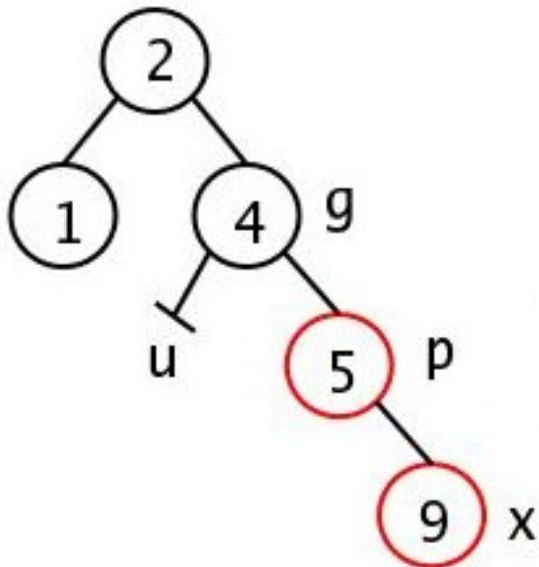
2 is root



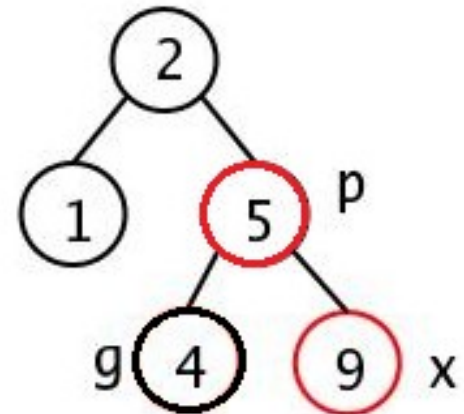
Example 2

Insert **2, 1, 4, 5, 9, 3, 6** and **7** in an empty Red-Black Tree.

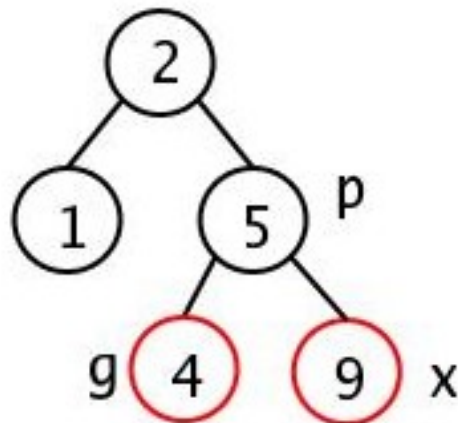
Insert(9)



step 1
(rotate)

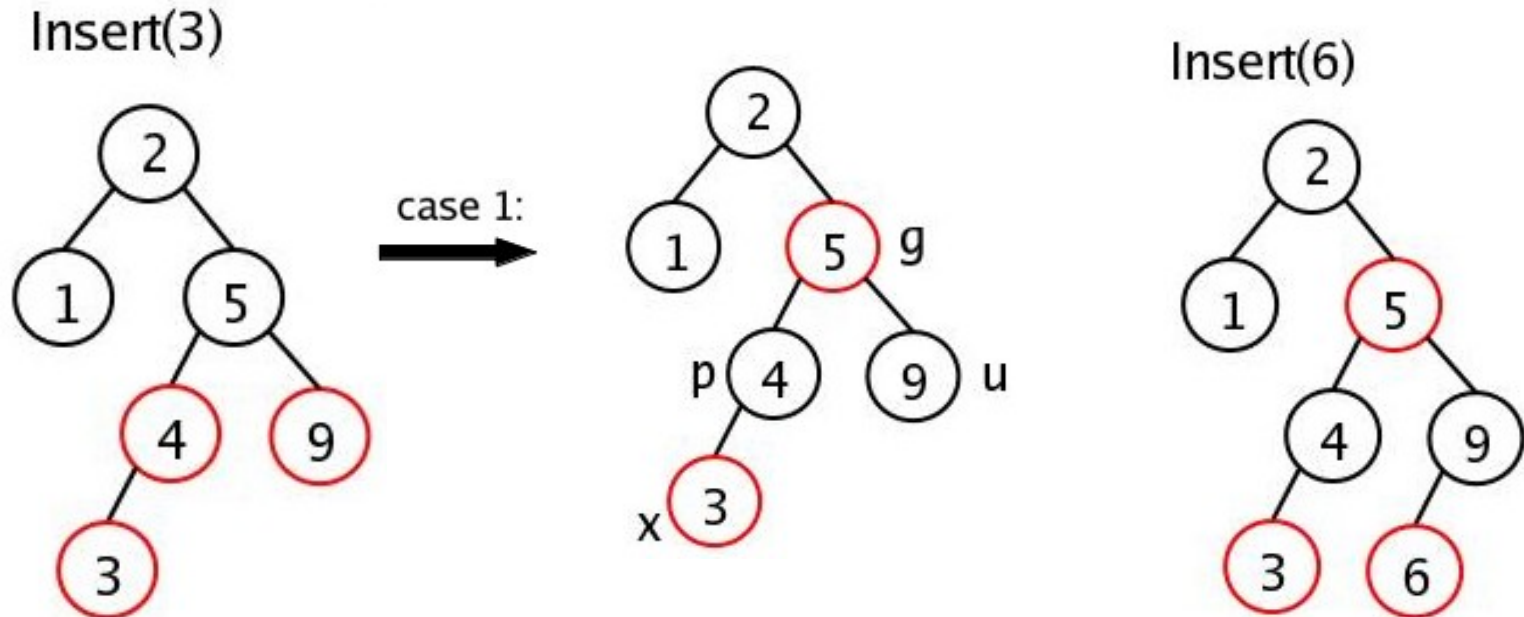


step 2
(recolor)



Example 2

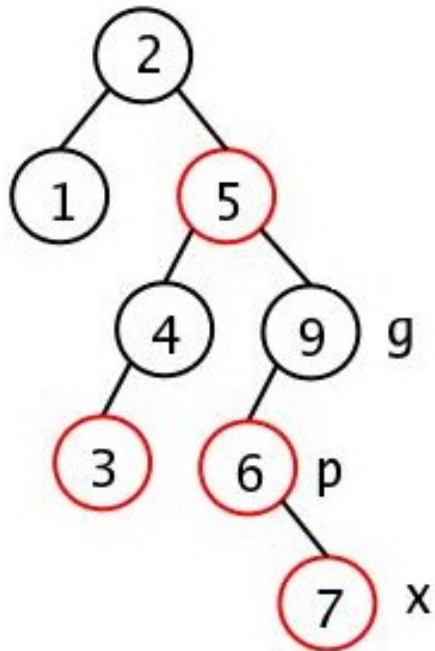
Insert **2, 1, 4, 5, 9, 3, 6** and **7** in an empty Red-Black Tree.



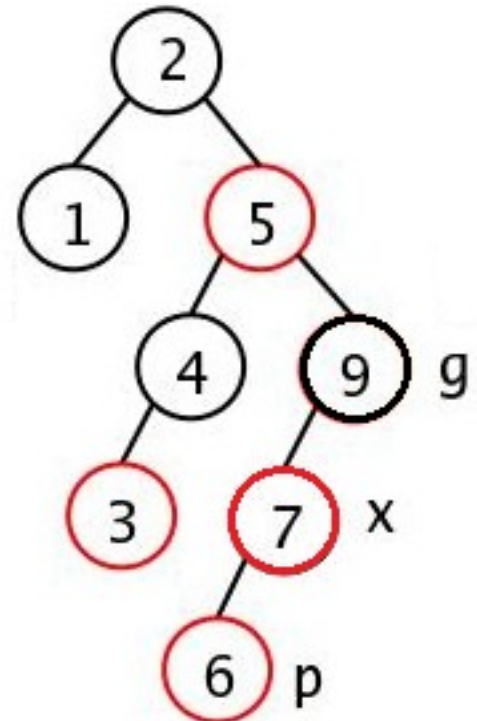
Example 2

Insert **2, 1, 4, 5, 9, 3, 6** and **7** in an empty Red-Black Tree.

Insert(7)

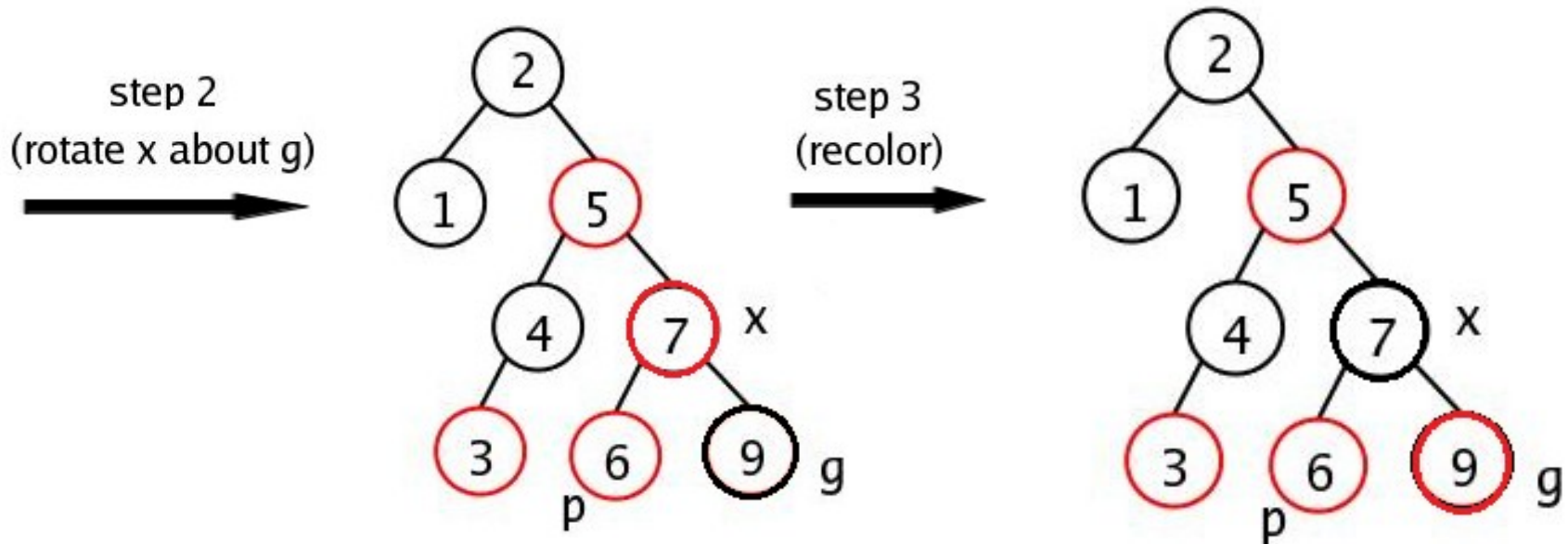


step 1
(rotate x about p)



Example 2

Insert **2, 1, 4, 5, 9, 3, 6** and **7** in an empty Red-Black Tree.



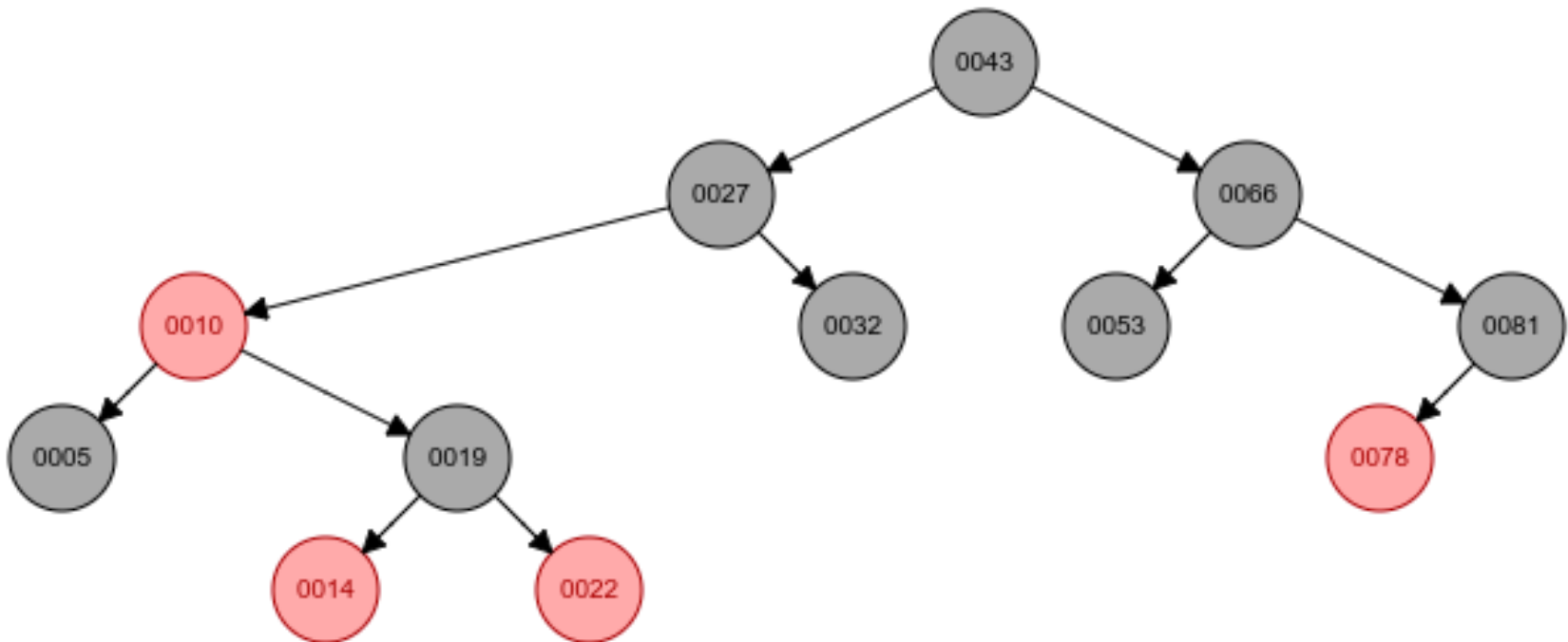
Assignment

Create a Red-Black Tree for the following entries, by inserting each number, picked in the order from left to right. Show each insertion separately with the diagram and mention the step that is performed.

Insert: **14, 27, 32, 10, 53, 5, 43, 81, 66, 78, 19, 22**

Solution to Assignment

Create a Red-Black Tree by inserting: **14, 27, 32, 10, 53, 5, 43, 81, 66, 78, 19, 22**



Deletion Operation

- Like Insertion, recoloring and rotations are used to maintain the Red-Black properties.
- In insert operation, color of uncle is checked to decide the appropriate case. In delete operation, **color of sibling is checked** to decide the appropriate case.
- The main property that violates after insertion is two consecutive reds. In delete, the main violated property is, change of black height in subtrees as deletion of a black node may cause reduced black height by one in root to leaf path.
- Deletion is fairly complex process. To understand deletion, notion of double black is used. When a black node is deleted and replaced by a black child, the child is marked as **double black**. The main task now becomes to convert this double black to single black.

Deletion Steps

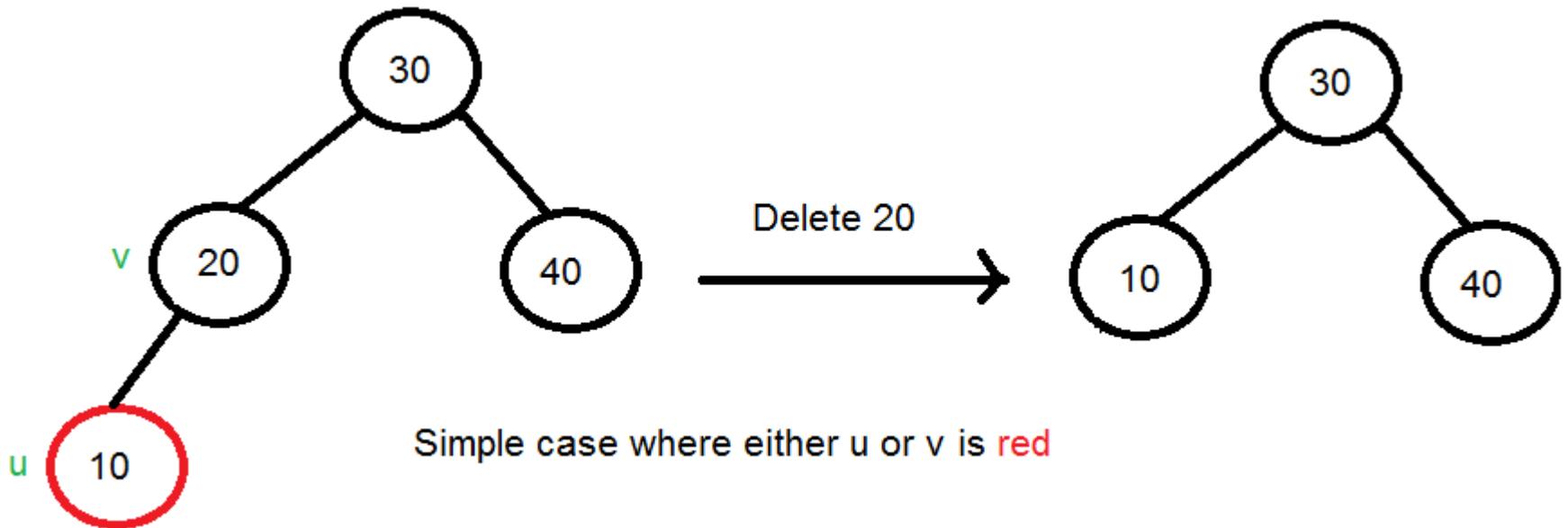
1. Perform standard BST delete.

- When standard delete operation is performed in BST, it always ends up deleting a node which is either leaf or has only one child (For an internal node, copy the successor and then recursively call delete for successor, successor is always a leaf node or a node with one child).
- So it is needed to handle only those cases where a node is leaf or has one child.
- Let ***v*** be the node to be deleted and ***u*** be the child that replaces ***v*** (Note that *u* is NULL when ***v*** is a leaf and color of NULL is considered as Black).

Deletion Steps

2. Simple case: If either u or v is red

- Mark the replaced child as black (No change in black height).
- Note that both u and v cannot be **RED** as v is parent of u and two consecutive red nodes are not allowed in red-black tree.

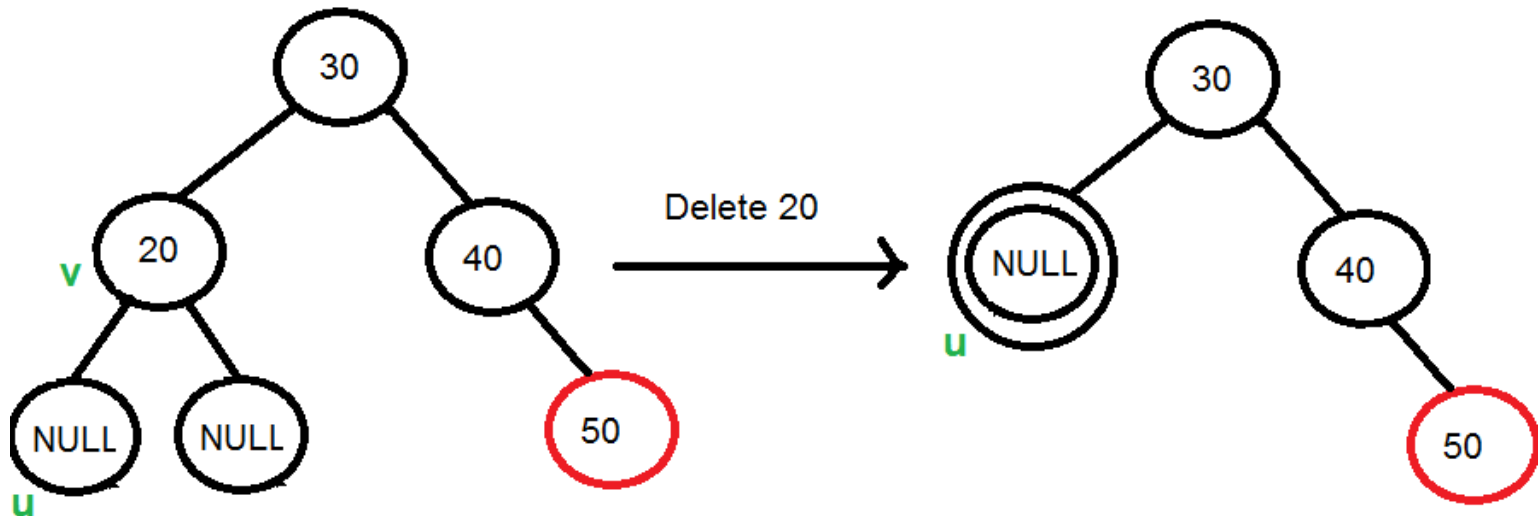


Deletion Steps

3. If both u and v is BLACK

3.1

- Color u as double black.
- Now task reduces to convert this double black to single black.
- Note that If v is leaf, then u is NULL and colour of NULL is considered as black. So the deletion of a black leaf also causes a double black.



When 20 is deleted, it is replaced by a NULL, so the NULL becomes double black.
Note that deletion is not done yet, this double black must become single black

Deletion Steps

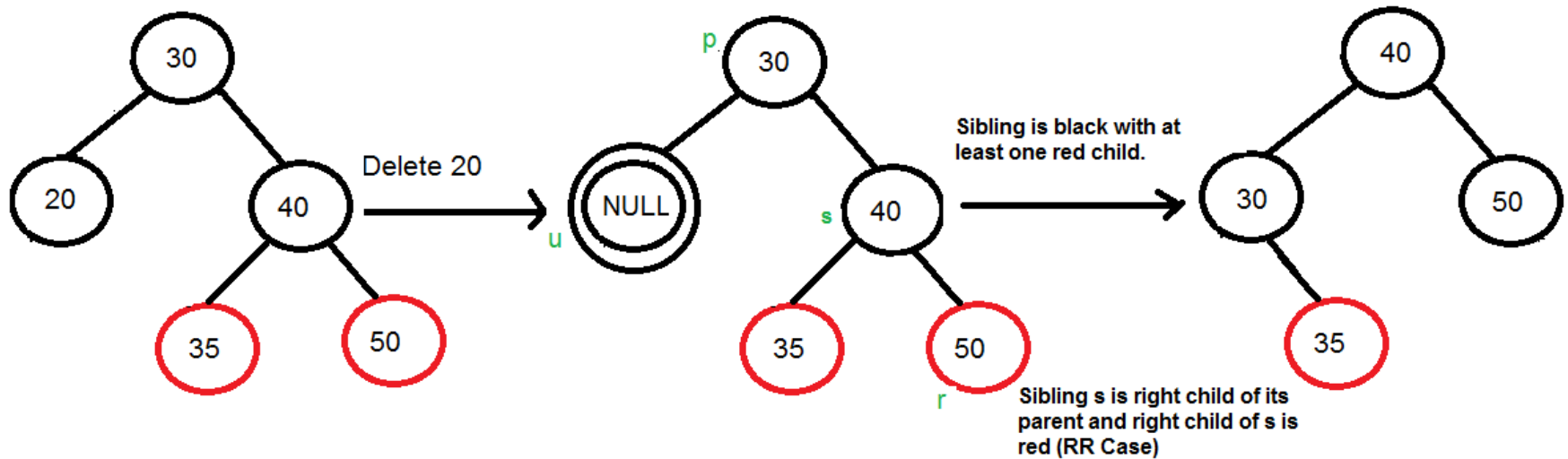
3.2 Do following while the current node u is double black and it is not root. Let sibling of node is s .

Case a:

- If sibling s is black and at least one of sibling's children is **red**, perform rotation(s).
- Let the red child of s be **r**. This case can be divided in four subcases depending upon positions of s and r .
 - **Left Left Case:** s is left child of its parent and **r** is left child of s or both children of s are red.
 - **Left Right Case:** s is left child of its parent and **r** is right child.
 - **Right Right Case:** s is right child of its parent and **r** is right child of s or both children of s are red.
 - **Right Left Case:** s is right child of its parent and **r** is left child of s

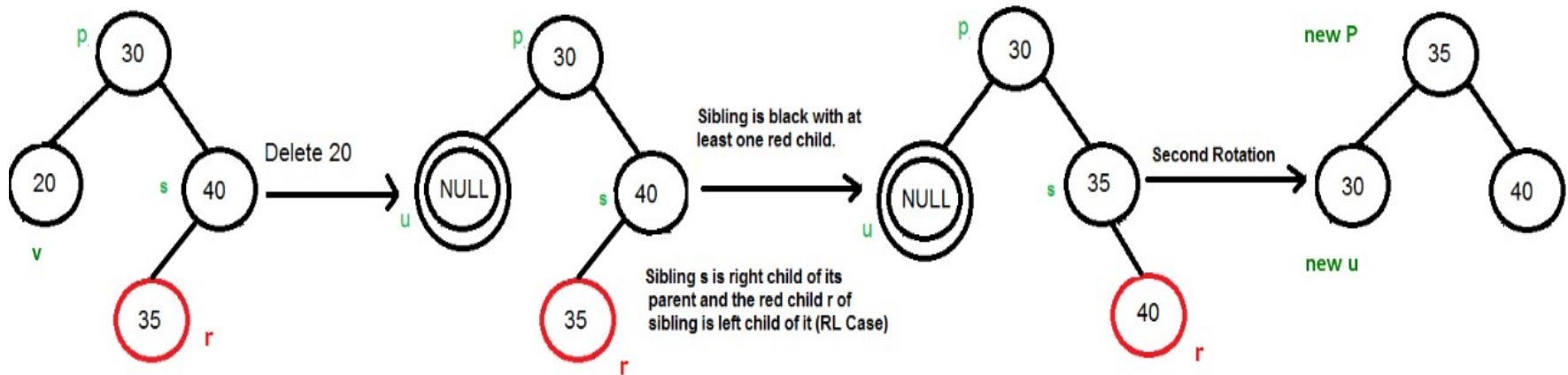
Deletion Steps

3.2 : Case a: Right Right Case



Deletion Steps

3.2 : Case a: Right Left Case

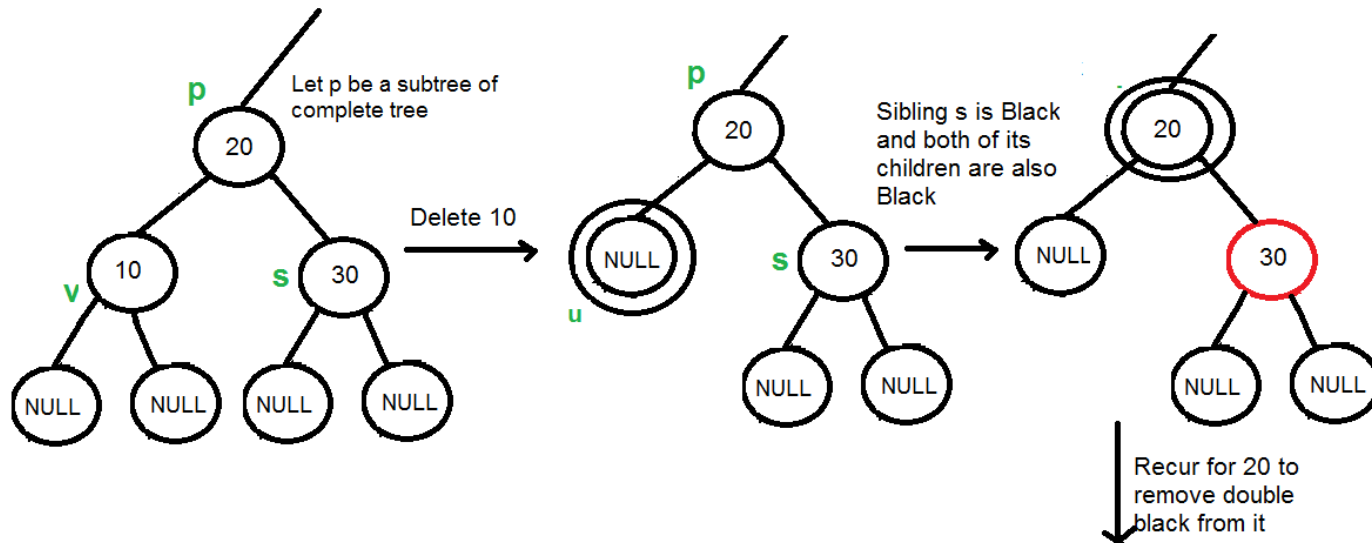


- After first rotation colors of s and r , are interchanged. Now it is reduced to right right case.
- After second rotation colors of new u and new p , are interchanged and color of r is changed to black.

Deletion Steps

3.2 Do following while the current node u is double black and it is not root. Let sibling of node is s .

Case b: If sibling s is black and its both children are black, then perform recoloring of u and s , and recur for the parent if parent is black.



- In this case, if parent was red, then we didn't need to recur for parent, we can simply make it black
- red + double black = single black

Deletion Steps

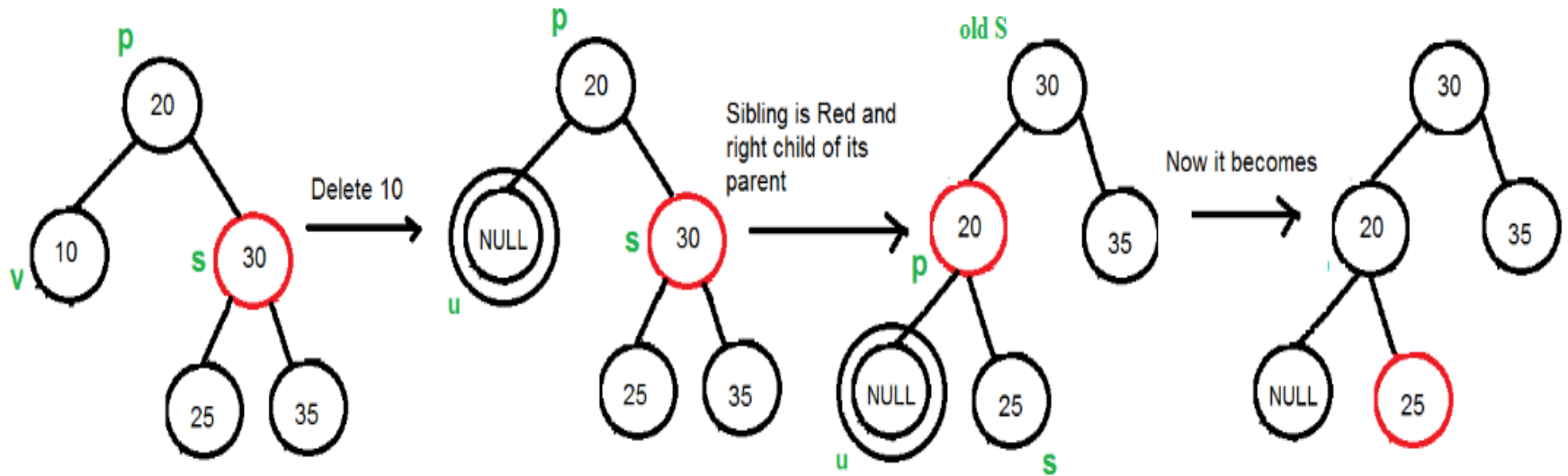
3.2 Do following while the current node u is double black and it is not root. Let sibling of node is s .

Case c: If sibling is **red**,

- Perform a rotation to move old sibling up, recolor the old sibling and parent.
- The new sibling is always black.
- This mainly converts the tree to black sibling case (by rotation) and leads to case (a) or (b).
- This case can be divided in two subcases.
 - **Left Case:** s is left child of its parent. Right rotate the parent p .
 - **Right Case:** s is right child of its parent. Left rotate the parent p .

Deletion Steps

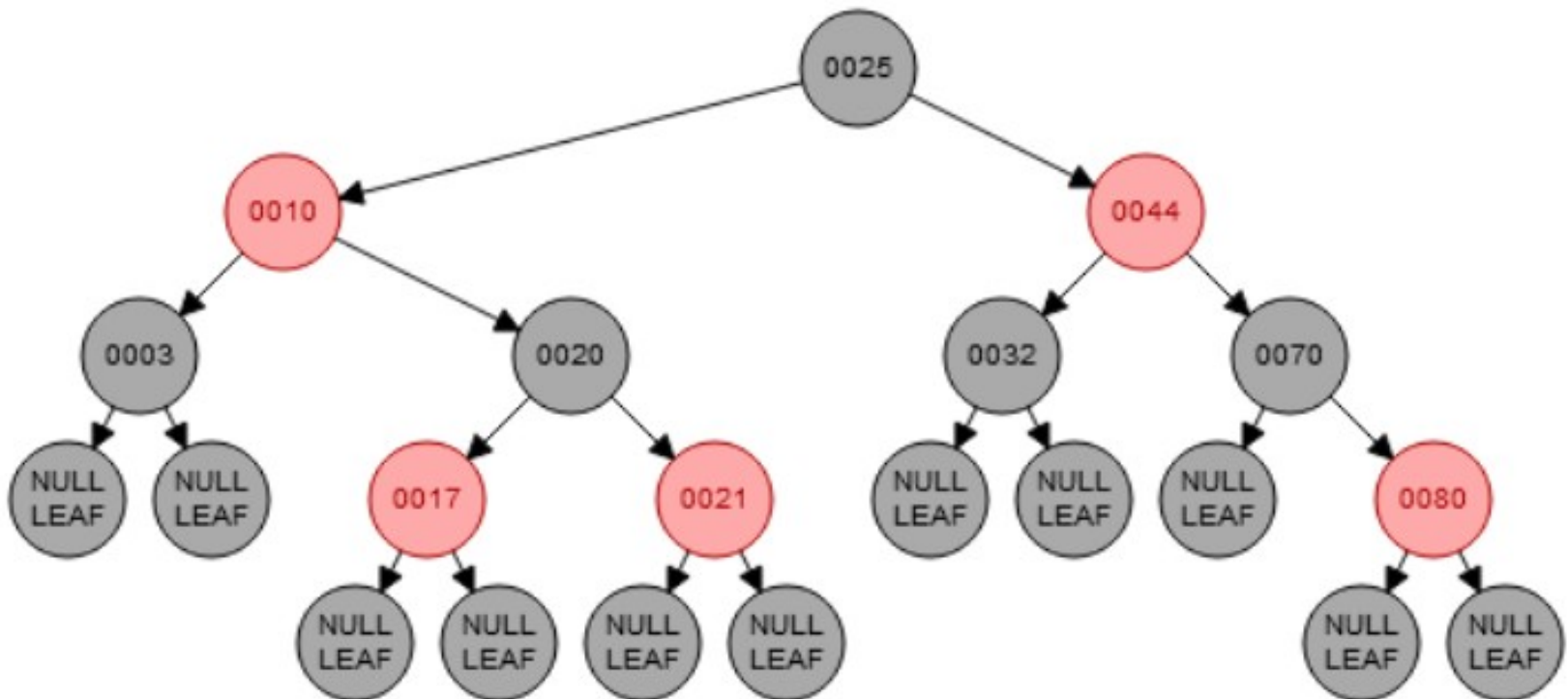
3.2 : Case c: Right Case



3.3 If **u** is root, make it single black and return. Black height of complete tree reduces by 1.

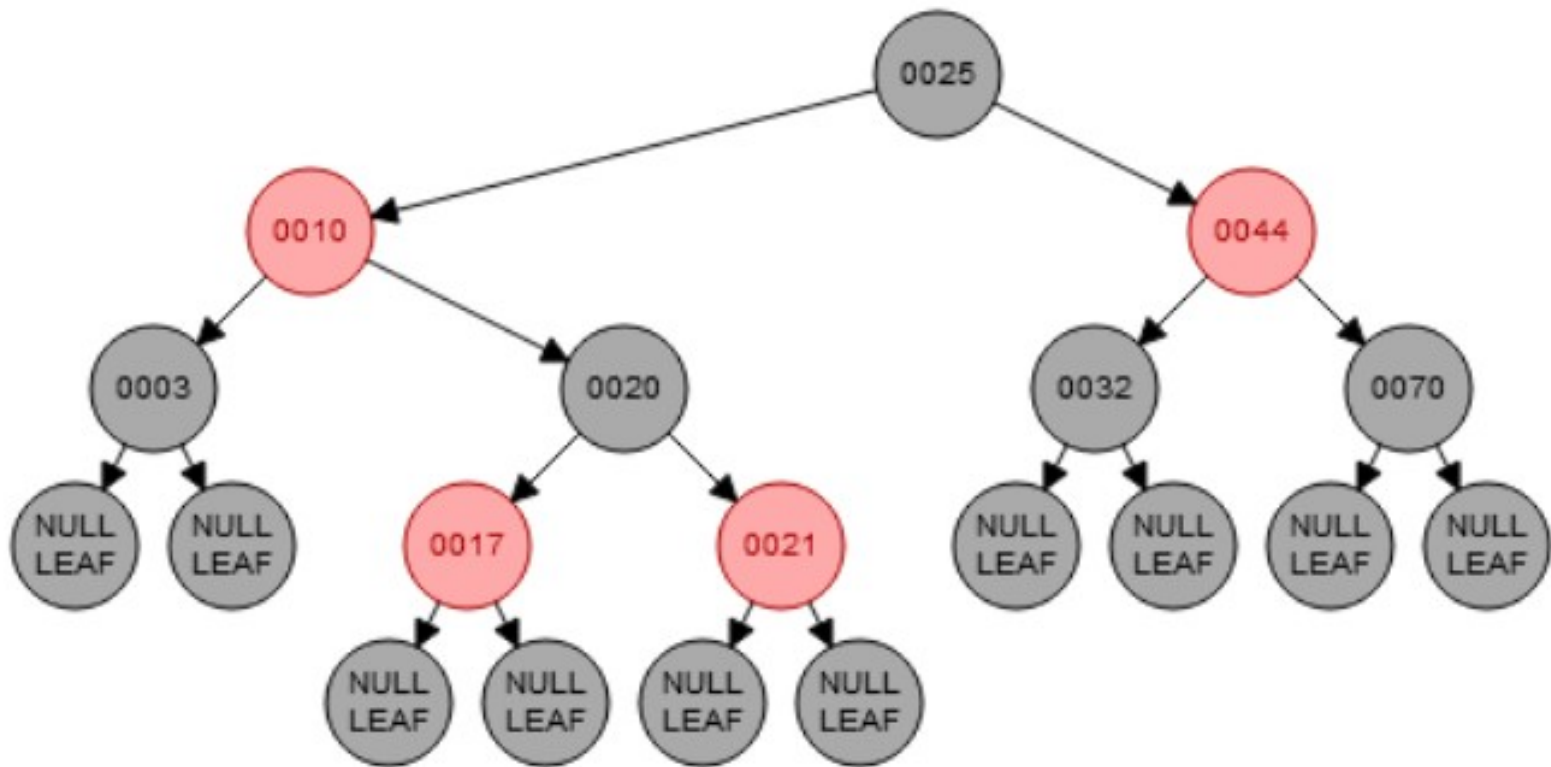
Example 3

Delete **80**, **20**, **32**, **21** and **3** from the following Red-Black Tree.



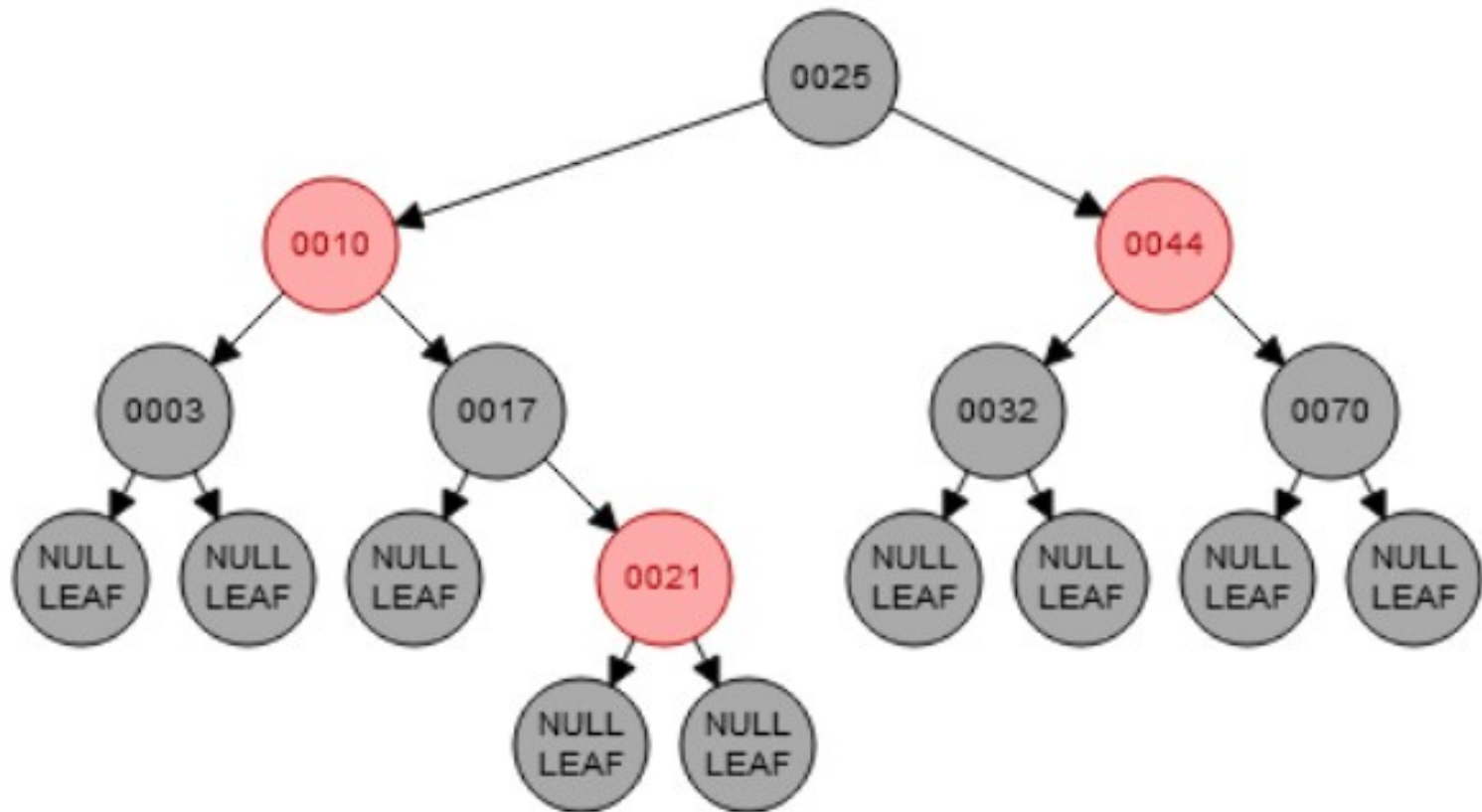
Example 3

Delete **80**: simple case



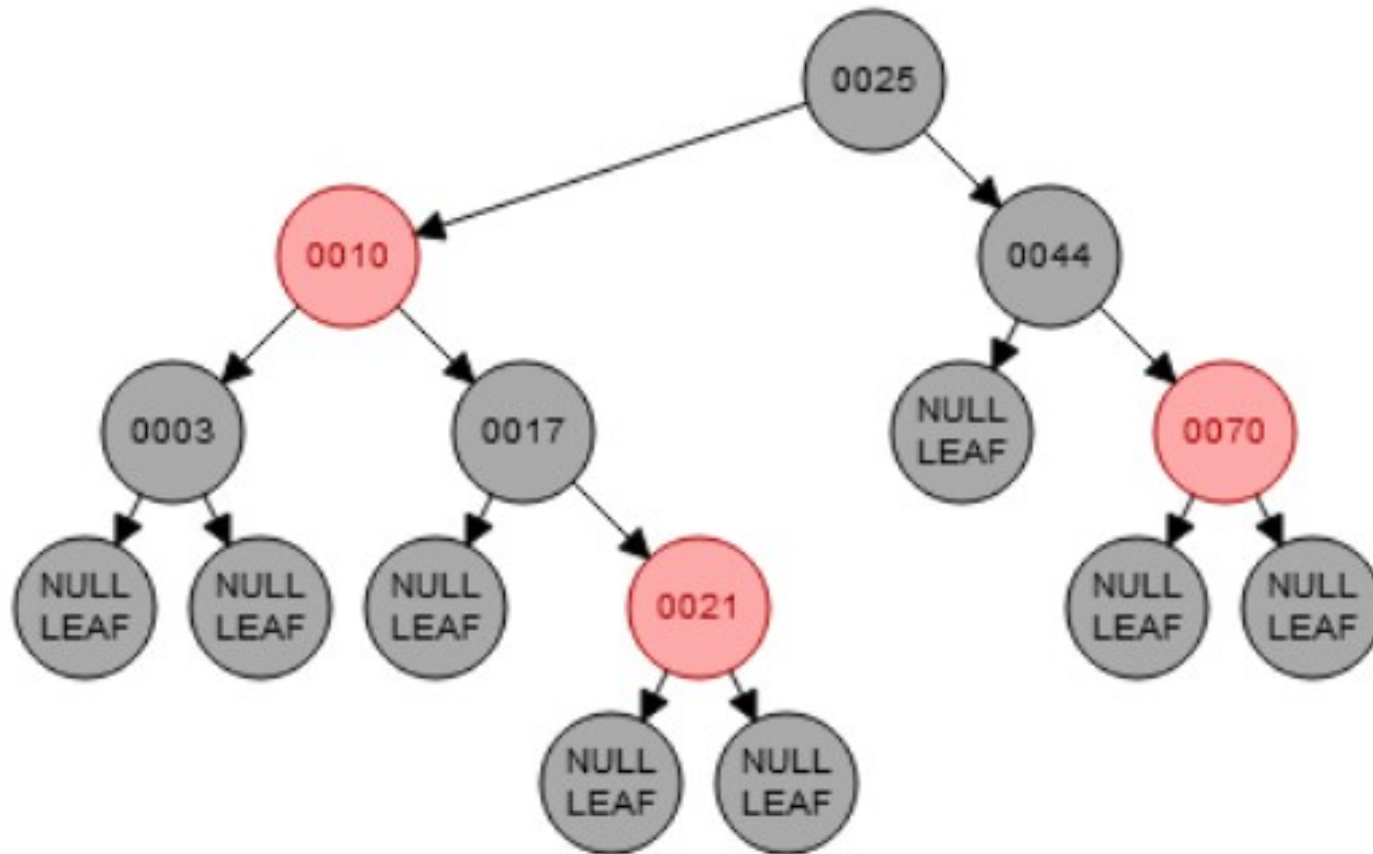
Example 3

Delete **20**: Node 20 will be replaced by predecessor 17 and 20 will be red leaf node, so deleted straight away



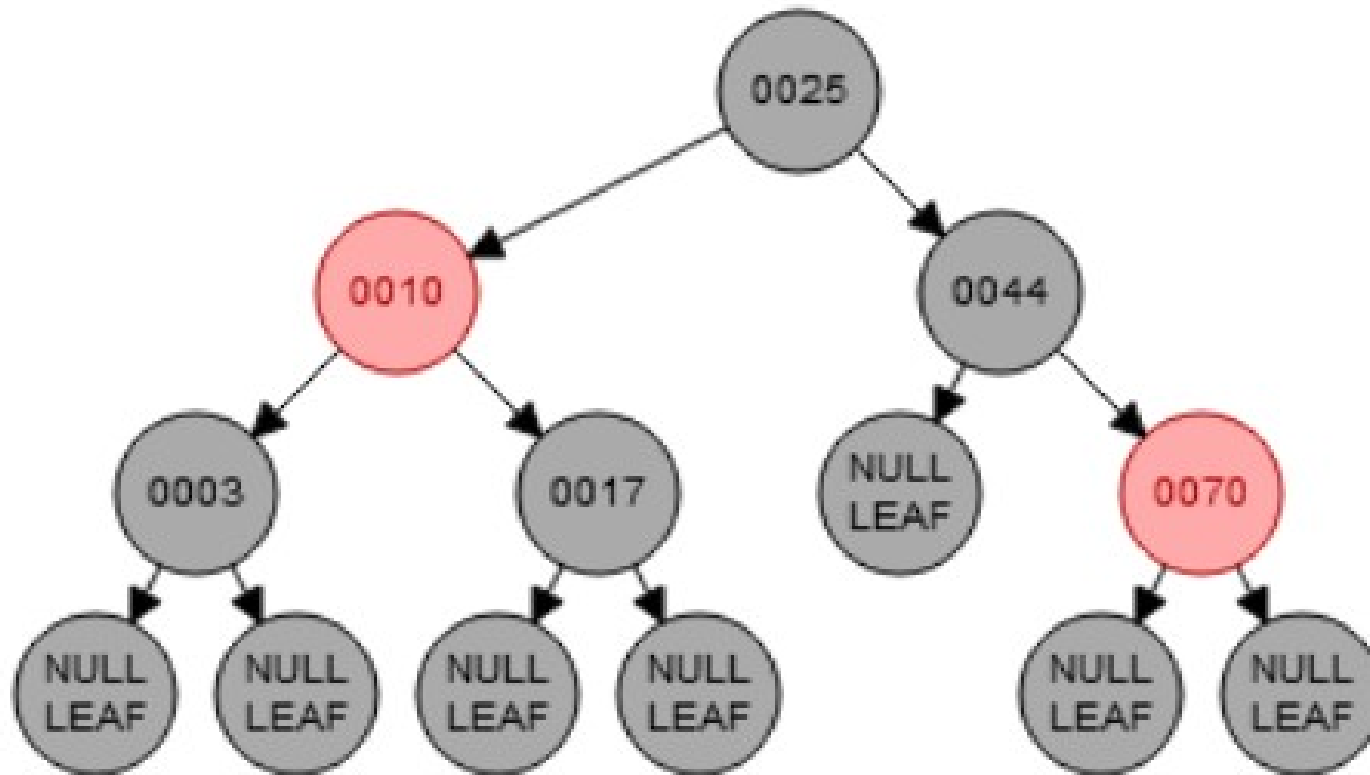
Example 3

Delete **32**: Case 3.2 b



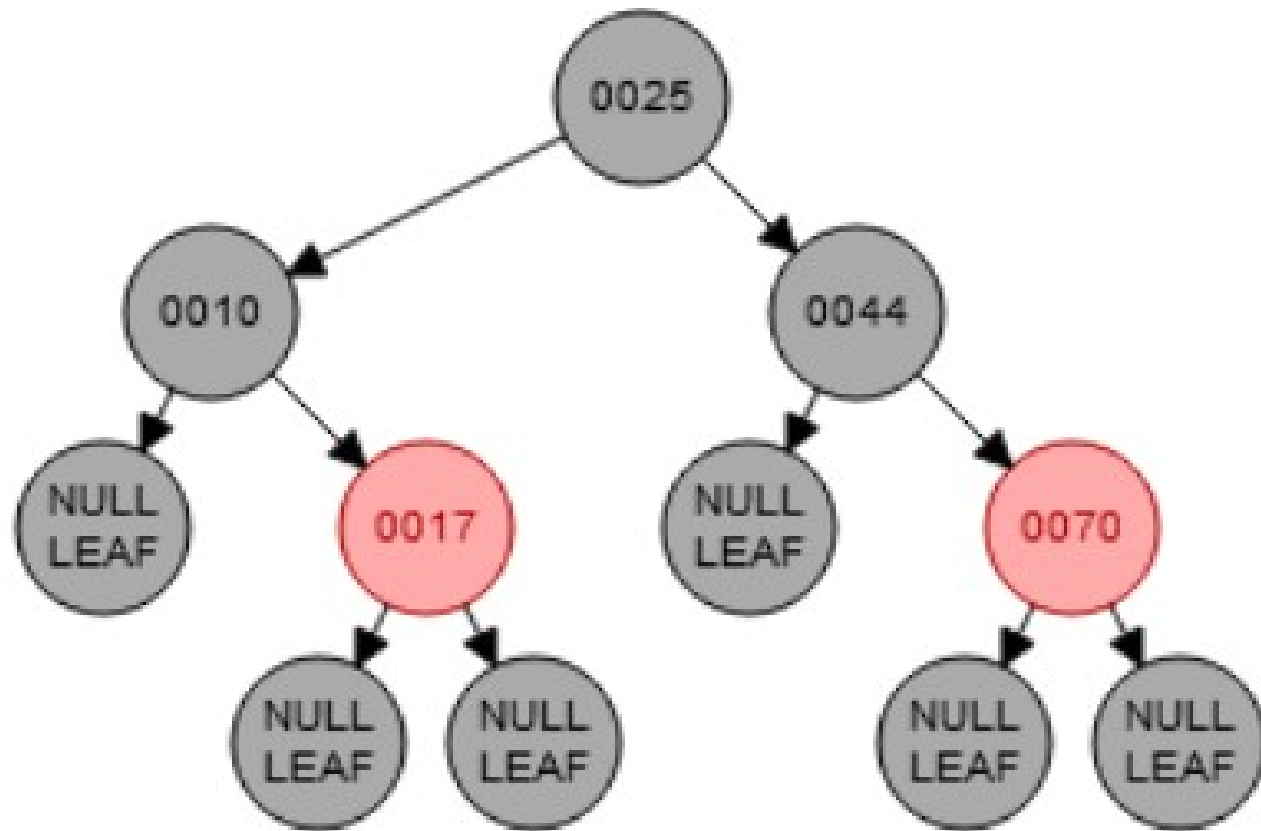
Example 3

Delete **21**: simple case



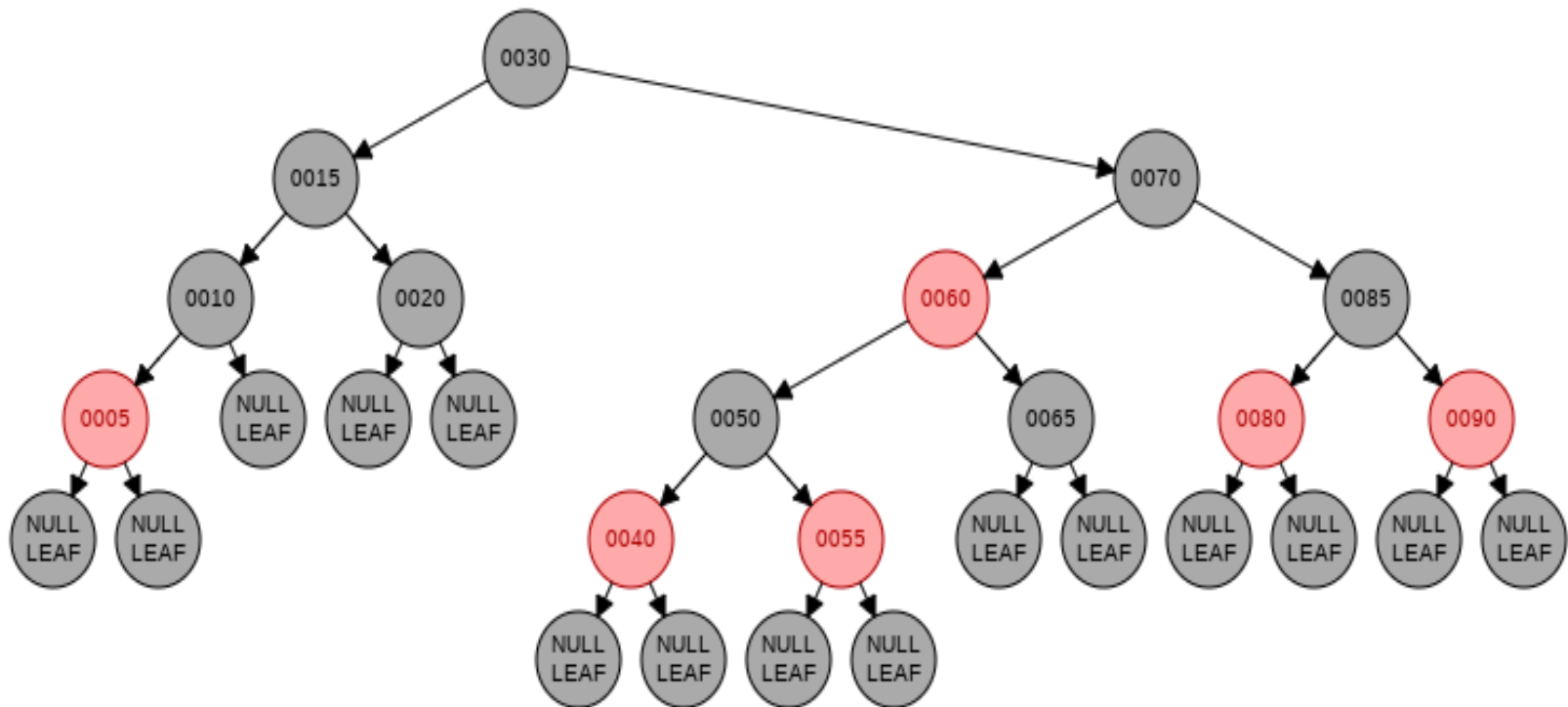
Example 3

Delete **3**: Case 3.2 b



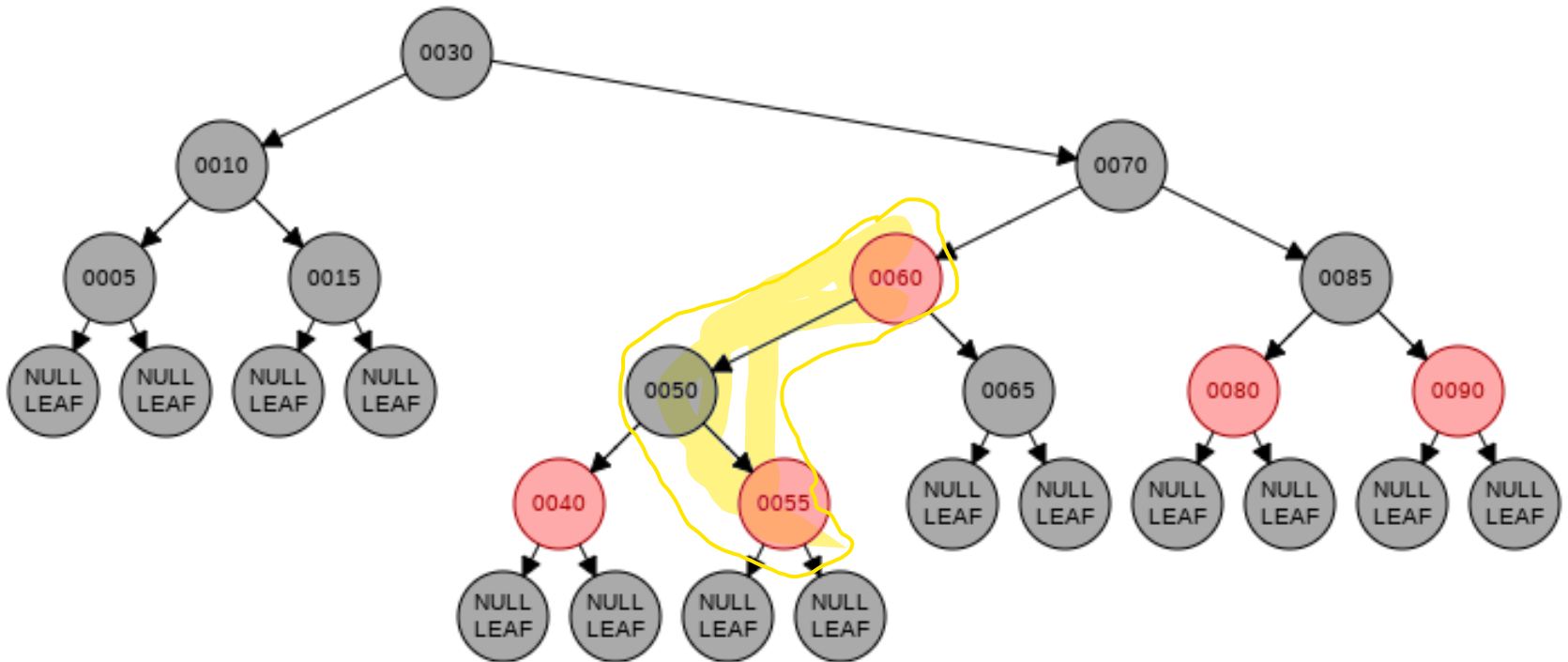
Assignment

Delete nodes **20, 60, 70, 10, 5, 90** from the below given Red Black Tree.



Solution to Assignment

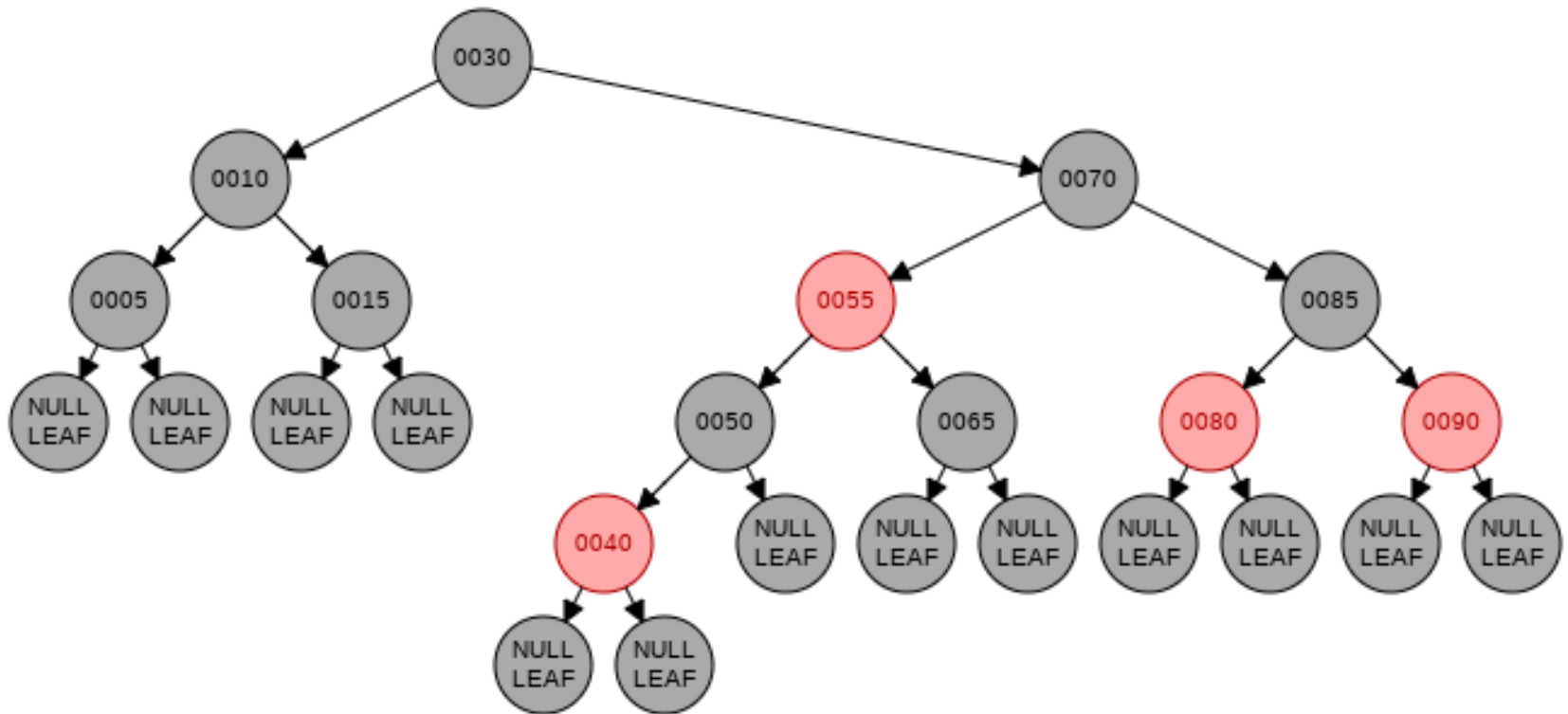
Delete **20**.



Next Delete **60**

Solution to Assignment

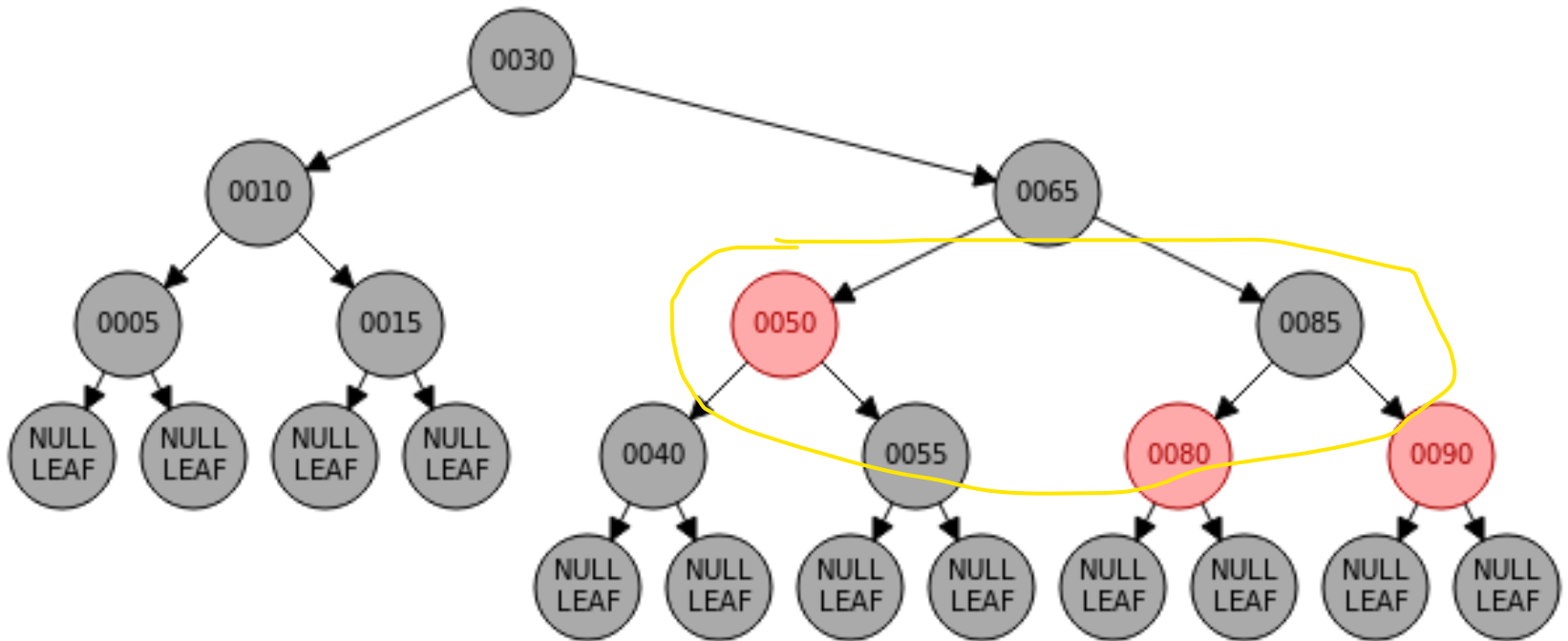
Delete **60**.



Next Delete **70**

Solution to Assignment

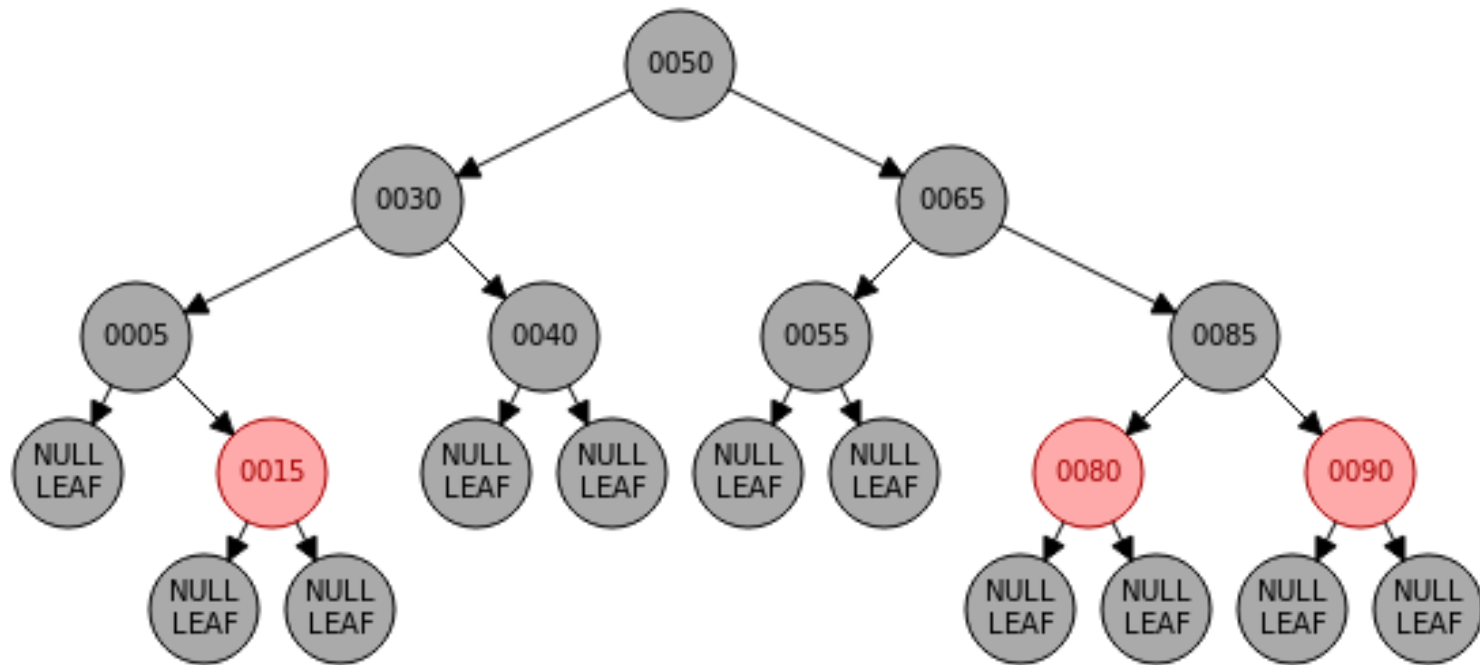
Delete **70**.



Next Delete **10**

Solution to Assignment

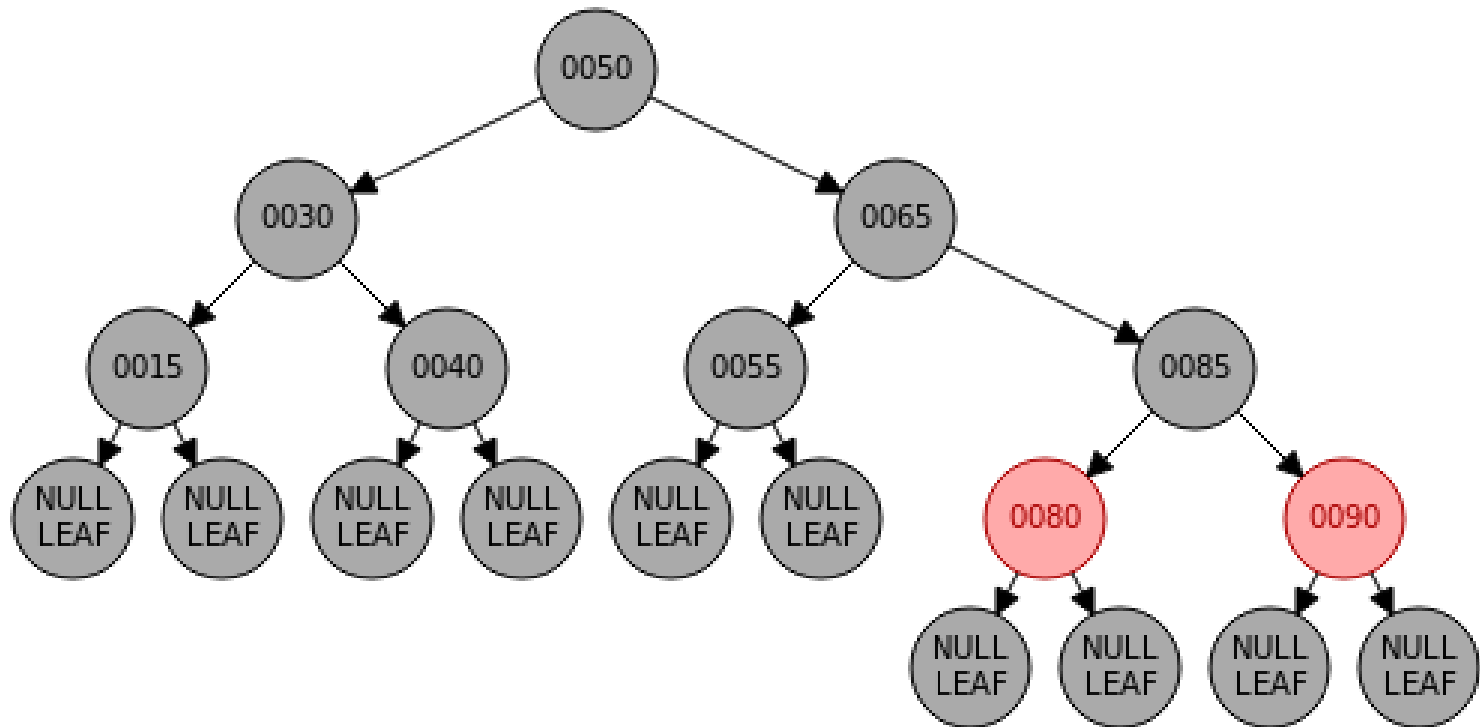
Delete **10**.



Next Delete **5**

Solution to Assignment

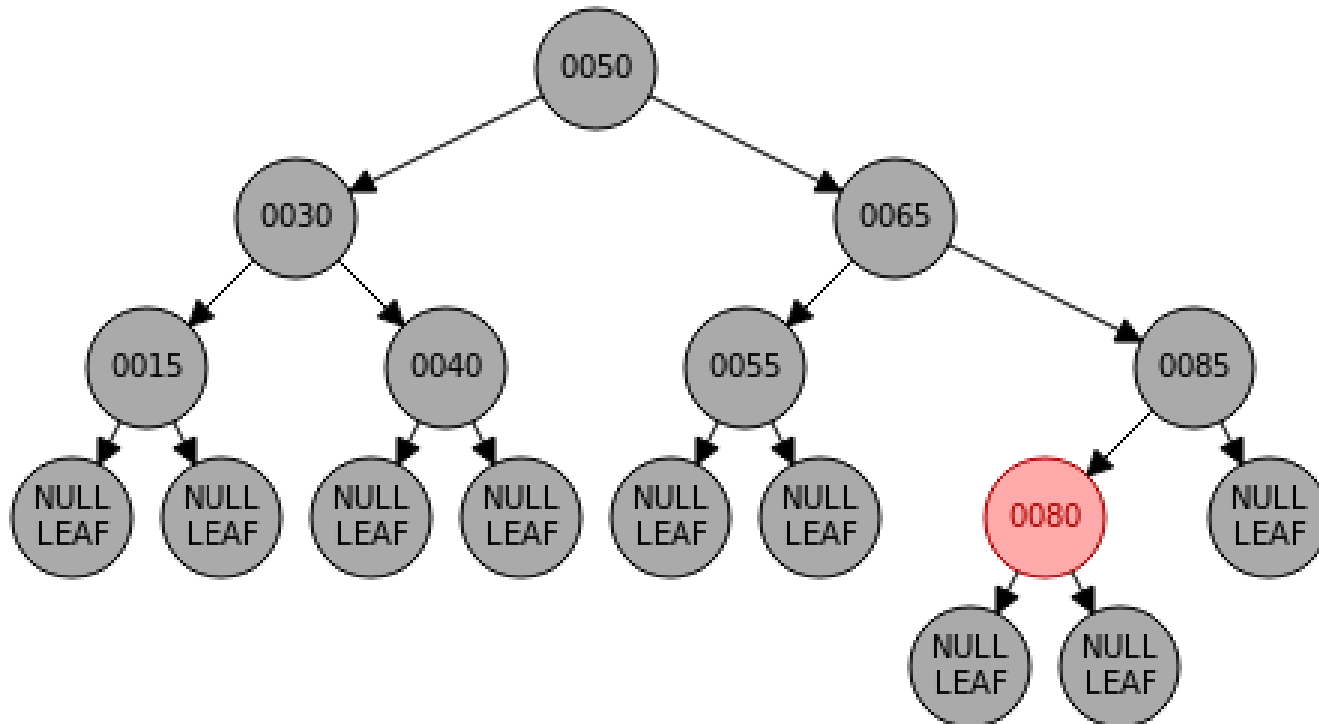
Delete **5**.



Next Delete **90**

Solution to Assignment

Delete **90**.



AVL vs RB Trees

- AVL trees provide faster lookups than Red Black Trees because they are more strictly balanced.
- RB Trees provide faster insertion and removal operations than AVL trees as fewer rotations are done due to relatively relaxed balancing.
- AVL trees store balance factors or heights with each node, thus requires storage for an integer per node whereas RB Tree requires only 1 bit of information per node.
- RB Trees are used in most of the language libraries like map, multimap, multiset in C++ whereas AVL trees are used in databases where faster retrievals are required.